

AI Text Classification Backend – Solution Walkthrough

1. Project Overview

Objective:
Build a Node.js backend with a single POST API to convert unstructured user text into a structured category (Complaint, Query, Feedback, Other). The result includes a category and a confidence score, using AI (OpenAI) for robust classification.

2. High-Level Architecture & Folder Structure

Layered Approach:

- **Client:** Uses Postman, curl, or a frontend to call API.
- **Express REST API:** Handles HTTP, validation, and routing.
- **Controller:** Validates requests, orchestrates business logic.
- **Service:** Calls AI, parses responses, handles fallback confidence logic.
- **Utils:** Utility functions such as confidence mapping.

Folder Structure:

```
ai-text-classifier/  
├── src/  
│   ├── controllers/  
│   │   └── classifyController.js  
│   ├── routes/  
│   │   └── classifyRoutes.js  
│   ├── services/  
│   │   └── aiClassifierService.js  
│   ├── utils/  
│   │   └── confidenceMapper.js  
│   ├── app.js  
│   └── server.js  
├── .env  
├── package.json  
├── README.md  
└── postman_collection.json
```

Why this structure?

- Separation of concerns (Easier maintenance, extensibility)
- Mimics industry standards
- Easily add features (explainability, auth, logging)

3. Tech Stack Choices

- **Node.js & Express:** Fast, popular backend framework.
- **dotenv:** To manage environment variables/configuration.
- **axios:** Safe, flexible HTTP client for OpenAI API.
- **OpenAI API:** Industry-leading large language model.
- **nodemon:** Improves developer workflow (auto restart).
- **Postman:** Documentation and manual testing.

4. API Design

POST /api/classify

• **Request:**

```
{ "text": "I am unhappy with your customer support" }
```

• **Response:**

```
{ "category": "Complaint", "confidence": 0.87 }
```

• **Error Example:**

```
{ "error": "Text is required" }  
{ "error": "AI service unavailable" }
```

Usage Example: (cURL)

```
curl -X POST http://localhost:3000/api/classify \  
-H 'Content-Type: application/json' \  
-d '{"text": "I am unhappy with your customer support"}'
```

5. AI & Prompt Engineering

Prompt Sent to LLM:

You are an **AI** text classification system.
Classify the given text into exactly **ONE** of the following categories:
Complaint
Query
Feedback
Other

Return **ONLY** a JSON object with:

```
{
  "category": "<one category>",
  "confidence": "<number between 0 and 1>"
}
```

Text:
"{USER_TEXT}"

Why this works well:
It forces a strict JSON response, exactly one category, with explicit confidence—which aids downstream parsing and = less code complexity.

6. Confidence Fallback

If the AI does NOT return confidence, this logic applies:

- Complaint → 0.85
- Query → 0.75
- Feedback → 0.80
- Other → 0.60

This is implemented in `src/utils/confidenceMapper.js`.

7. Configuration

.env file example:

```
PORT=3000
OPENAI_API_KEY=your_openai_api_key_here
OPENAI_MODEL=gpt-3.5-turbo
INCLUDE_DEBUG=false
```

How to run:

1. `npm install`
2. Create `.env` as above.
3. `npm run dev`
4. Call `/api/classify` with JSON.

8. Extensibility & Best Practices

- Modular structure allows:
 - New models (Gemini, local logic)
 - Explainability or text length limit
 - Auth, rate limiting, metrics, etc.
- All logic (AI, confidence, controller) is separated and unit-testable.
- API docs/test collection included (Postman).

9. Future Improvements (Optional Suggestions)

- Add explanations:

```
{
  "category": "Complaint",
  "confidence": 0.87,
  "reason": "Negative sentiment and keywords"
}
```

- Rate limiting/throttling for production
- Dockerize the app for deployment

10. Author & Philosophy

- Designed for clarity, maintainability, and extensibility.
- Favors industry structure over code golfing or shortcuts.

End of Explanation